

umm API & MCP 연동 가이드 (API & MCP Guide)

umm은 시스템 통합 및 자동화를 위한 **REST API**와 AI 에이전트(Claude Desktop, Cursor, Custom Agent 등) 연동을 위한 **Model Context Protocol (MCP)** 엔드포인트를 제공합니다.

1. 인증 체계 (Authentication)

모든 API 및 MCP 호출은 개인 설정(`/settings`)에서 발급받은 **Bearer API Key** 또는 세션 쿠키를 사용합니다:

```
Authorization: Bearer umm_key_a1b2c3d4_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

2. REST API 주요 엔드포인트 (`/api/v1`)

공간 (Spaces)

메서드	경로	설명	권한 스코프
GET	<code>/spaces</code>	사용자가 접근 가능한 공간 목록 조회	<code>spaces:read</code>
POST	<code>/spaces</code>	새 공간 생성 (<code>{"name": "새 프로젝트"}</code>)	<code>notes:write</code>
PUT	<code>/spaces/{id}</code>	공간 이름 변경 (<code>{"name": "수정된 이름"}</code>)	<code>notes:write</code>
DELETE	<code>/spaces/{id}</code>	공간 및 내부 메모 영구 삭제	<code>notes:write</code>
GET	<code>/spaces/{id}/members</code>	공간 참여 멤버 및 권한 목록 조회	<code>spaces:read</code>
POST	<code>/spaces/{id}/members</code>	공간에 사용자 초대	<code>notes:write</code>

생각 포스트잇 (Notes & Edges)

메서드	경로	설명	권한 스코프
GET	/spaces/{id}/notes	공간 내 모든 포스트잇과 연결선 조회	notes:read
POST	/spaces/{id}/notes	새 포스트잇 생성 (title , content , color , x , y , width , height)	notes:write
PUT	/notes/{id}	포스트잇 내용/위치/크기 수정	notes:write
DELETE	/notes/{id}	포스트잇 삭제	notes:write
GET	/notes/{id}/history	포스트잇 과거 수정 이력 스냅샷 조회	notes:read
POST	/notes/{id}/restore/{version}	특정 버전으로 포스트잇 복원	notes:write
GET	/notes/{id}/related	의미 유사도 기반 연관 생각 목록 추천	notes:read
GET	/notes/{id}/backlinks	들어오고 나가는 실제 연결선과 상대 생각 조회	notes:read
GET	/search?q= ...	로컬 의미·키워드 하이브리드 검색과 필터 ·cursor	notes:read
GET	/notes/{id}/comments	댓글·멘션 목록	notes:read
POST	/notes/{id}/comments	댓글 작성과 멘션 알림	notes:write
PUT	/comments/{id}/resolve	댓글 해결·재개	notes:write
POST	/spaces/{id}/edges	두 포스트잇 간 연결선 생성 (source , target , relation)	notes:write

AI Assist

메서드	경로	설명	권한 스코프
POST	/ai/assist	선택한 1~20개 생각을 요약·질문·확장·반대 관점·실행 항목으로 발전	ai:assist

ai:assist 는 선택한 본문을 외부 Gateway로 보내고 AI 쿼터를 소비하는 기능만 따로 허용하는 최소 권한 scope입니다. **notes:read** 만 가진 API key는 일반 생각 조회를 할 수 있어도 AI Assist를 호출할 수 없습니다. 서버는 선택한 모든 생각이 현재 접근 가능하고 AI 제외 상태가 아닌지 외부 호출 직전에 다시 확인합니다. 해당 `note-space-membership` 행을 Gateway 응답이 끝날 때까지 잠그므로 공유 회수나 AI 제외가 먼저 확정되면 본문을 보내지 않고 미사용 쿼터를 취소하며, AI lease가 먼저 시작되면 정책 변경은 호출 종료까지 기다립니다. 긴 lease는 일반 요청 pool과 분리된 인스턴스당 최대 2개 PostgreSQL 연결만 사용하므로, 추가 AI 호출이 대기해도 readiness-인증·Canvas용 `pool_max_conns` 를 점유하지 않습니다.

메서드	경로	설명	권한 스코프
GET	/today	검토 대상·고립 생각·댓글·온보딩 집계. Dream 항목은 별도 권한이 있을 때만 포함	notes:read (dreams:read for Dream)
POST	/notes/{id}/review	검토 완료, 다시 보기, 고정	notes:write
GET	/onboarding	실제 사용 기반 온보딩 진행률	로그인
POST	/onboarding/complete	온보딩 완료 표시	로그인

🌙 Dream 검토함

메서드	경로	설명	권한 스코프
GET	/dreams	출처와 상태를 포함한 개인 Dream 검토함 조회	dreams:read
POST	/dreams/{id}/feedback	노출·채택·숨김 등의 무중복 피드백 기록	dreams:read
POST	/dreams/{id}/accept	후보를 Dream 메모와 출처 연결선으로 확정	dreams:read , notes:write
POST	/dreams/{id}/regenerate	같은 출처에서 중복되지 않는 다른 관점 생성	dreams:read
POST	/dreams/{id}/develop	확장·반대 관점·실행 항목으로 발전	dreams:read
POST	/dreams/{id}/developed-note	발전 결과를 새 메모와 expanded 연결선으로 원자 저장(동일 재시도 무중복)	dreams:read , notes:write

Today의 대기 Dream과 GET /dreams 의 후보·출처는 매 조회마다 Dream이 속한 공간의 현재 owner/member 권한을 확인합니다. 자동 Dream의 원본 선별도 현재 접근 가능한 사용자 작성 생각만 허용하고, 최종 source lease 안에서 자격을 다시 확인합니다. 공간 공유가 회수되면 파생 본문·공간명·원본이 즉시 목록에서 빠지고 보관한 Dream ID로 피드백·숨김·채택·재생성·발전을 요청해도 상태나 개인화를 바꾸지 않습니다. 피드백은 Dream 행과 membership을 같은 transaction에서 잠급니다. 자동 생성·재생성·발전은 AI 제외되지 않은 원본 note, 관련 공간과 현재 membership을 정렬된 순서로 잠그며 Dream 기반 호출은 Dream 행과 공간도 포함한 뒤 그 lease 안에서만 Gateway를 호출합니다. 회수가 먼저 끝나면 외부 호출 전 쿼터 예약을 취소하고 본문을 보내지 않으며, lease가 먼저 시작되면 회수 transaction이 호출 종료 뒤까지 기다립니다. 자동 생성 후보·source·알림과 채택·발전 결과 저장도 각 열린 transaction 안에서 함께 확정합니다. 별도 pool 연결로 권한을 다시 조회하지 않으므로 작은 pool에서도 자신이 보유한 transaction 연결을 기다리지 않습니다.

GET /notifications 의 각 항목에는 resourceType , resourceId , resourceSpaceId , metadata 가 포함됩니다. Dream 알림은 /dreams?focus={resourceId} , 공간 공유 알림은 /space/{resourceId} , 댓글·멘션은 /space/{resourceSpaceId}?note={resourceId} 로 연결할 수 있습니다. note 알림은 현재 생각의 soft-delete 상태와 실제 공간 접근권한을 목록·unread count 양쪽에서 확인하며, 이전 행에 resourceSpaceId 가 없어도 생각이 속한 공간으로 권한을 계산합니다. Dream 알림도 현재 공간 ID를 저장하고 이전 행은 Dream에서 실제 공간을 찾아

공유 회수 뒤 본문을 숨깁니다. unread count와 page 조회는 DB 연결을 겹쳐 잡지 않고 순차 실행합니다.

`pool_max_conns` 1~2에서는 realtime listener도 안전 폴링으로 전환하므로 endpoint가 자신이 필요한 request 연결을 기다리지 않습니다.

개인 설정과 API key 생성·회전·폐기, 세션 관리, 모든 `/admin/*` 작업은 API key가 아니라 로그인한 브라우저 세션에서만 허용됩니다. 제한된 자동화 key가 새 자격 증명을 만들거나 관리자 role을 상속해 권한을 넓힐 수 없습니다. OIDC·AI Gateway 설정에서 마스킹된 secret을 그대로 보내면 서버가 저장 직전의 최신 암호문을 설정별 transaction lock 안에서 병합하며, master-key 회전도 같은 lock을 사용하므로 회전 전에 연 관리 화면의 저장에 새 암호문을 되돌리지 않습니다.

`PUT /preferences` 는 원자적 부분 수정입니다. 바꾸려는 필드만 보내면 되고, 보내지 않은 필드는 요청이 처리되는 시점의 최신 저장값이 유지됩니다. 언어와 테마처럼 서로 다른 설정을 동시에 바뀌도 한 요청이 다른 요청을 되돌리지 않으며, `dream_pause_until: null` 은 일시정지를 명시적으로 해제합니다.

🔑 로그인한 기기

메서드	경로	설명	권한
GET	<code>/sessions</code>	이 계정에 로그인된 브라우저 세션 목록. 요청을 보낸 세션은 <code>current</code> 로 표시	브라우저 세션
DELETE	<code>/sessions/{id}</code>	특정 세션 종료. 현재 세션을 종료하면 쿠키도 함께 지워짐	브라우저 세션
POST	<code>/sessions/revoke-others</code>	이 브라우저를 제외한 모든 로그인 종료	브라우저 세션

세션 토큰은 어떤 응답에도 포함되지 않습니다.

세션의 `userAgent` 는 표시·감사용 정보이며 서버가 유효한 UTF-8의 최대 300 byte로 정규화합니다. 다중 byte 문자는 rune 경계에서만 잘리므로 긴 User-Agent 때문에 올바른 로그인 자체가 실패하지 않습니다.

Canvas 목록·수정 이력·댓글·백링크처럼 본문을 반환하는 조회는 사전 권한 확인 결과를 재사용하지 않고, 콘텐츠를 읽는 PostgreSQL statement 자체에서 현재 공간 owner/member 조건을 확인합니다. 멤버가 제거된 뒤 시작된 조회는 빈 본문을 반환하지 않고 404로 종료됩니다.

댓글 작성은 생각과 공간을 확인한 뒤 호출자의 현재 멤버십 행을 transaction 종료까지 잠급니다. 작성과 권한 회수가 겹치면 먼저 잠금을 얻은 작업이 끝난 뒤 다음 작업이 진행되며, 권한 회수가 먼저 확정된 요청은 댓글·알림·실시간 이벤트·webhook outbox를 만들지 않고 404로 종료됩니다. 생각이 soft-delete되었거나 공간에서 제거된 뒤 보관한 댓글 ID로 해결·재개·삭제를 요청하면 현재 상태를 mutation statement에서 확인하고 `comment-mutation-forbidden` 403 Problem Details로 종료해 행·이벤트·webhook을 바꾸지 않습니다. 이 타입은 재시도해도 적용할 수 없는 오프라인 변경임을 뜻합니다. `review_digest=false` 는 `/today` 의 협업 활동만 제외하고 고정·다시 보기 기한·오래된 생각의 기본 검토 목록은 유지합니다.

`/search` , `/notifications` , `/dreams` , `/admin/audit` 는 응답의 `nextCursor` 를 다음 요청의 `cursor` 에 그대로 전달합니다. cursor 내부 형식에 의존하지 마세요.

하이브리드 검색은 접근 가능한 전체 메모에서 정확 제목·정확 본문·제목 구문·본문 구문 순으로 키워드 후보를 제한 한 뒤 최근 비키워드 후보에 의미 점수를 결합합니다. 따라서 광범위한 단어가 500개 넘는 본문에 포함되거나, 메모가 2,000개를 넘거나, 검색어가 본문 2,000자 이후에 있어도 오래된 정확 결과를 최신 부분 일치 때문에 누락하지 않습니다.

키워드 조건은 `pg_trgm` 인덱스를 사용하며 검색어는 최대 8개 단어까지 반영됩니다(그 이상은 무시되어 결과가 좁아지지 않고 넓어집니다). 한 검색어의 모든 단어는 메모 본문 쪽 또는 공간 이름 쪽 한 곳에서 모두 일치해야 하며, 양 쪽에 나뉘어 걸친 조합은 의미 점수 후보로 넘어갑니다.

요청 한도

상황	응답	problem type
호출자별 API 요청 한도 초과	429 + Retry-After	rate-limited
AI 생성 분당 한도 초과	429 + Retry-After	ai-rate-limited
AI 생성 하루 한도 초과	429 + Retry-After	ai-daily-limit
AI 쿼터 저장소 일시 오류	503 + Retry-After	ai-quota-unavailable
로그인 실패 반복으로 잠금	429 + Retry-After	login-locked

한도 값은 관리자 설정에서 조정합니다. 구버전 관리 클라이언트가 새 남용 방지 필드를 생략한 채 보안 섹션을 PUT 해도 서버는 PostgreSQL transaction lock 아래 최신 저장값에서 생략 필드만 병합합니다. 명시한 0은 하루 AI 무제한으로 적용되고 null은 잘못된 설정으로 거부됩니다. 로컬 로그인은 정렬된 주소·계정 advisory lock 아래 잠금 확인부터 비밀번호 검증과 실패 기록 또는 session 저장까지 같은 PostgreSQL 연결에서 끝내므로, 여러 replica의 병렬 요청도 login_max_failures 보다 많은 추측을 통과시키지 않고 작은 pool에서도 추가 연결을 기다리지 않습니다. 일일 AI 한도는 실제 Gateway 호출 전에 PostgreSQL에서 원자적으로 예약한 뒤 24시간 원장으로 영속화하며, 자동 Dream과 관리자 AI 평가도 같은 원장을 사용합니다. 따라서 replica가 여러 개거나 요청이 취소되거나 별도 ai_calls 로그 저장이 실패해도 동시에 한도를 넘지 않습니다. 쿼터를 영속화할 수 없으면 비용 보호를 위해 AI 호출을 시작하지 않고 503으로 종료합니다. 429와 503은 재시도 가능한 응답이므로 오프라인 queue도 보존한 뒤 다시 시도합니다.

서명 웹훅

/webhooks에서 이벤트 subscription을 만들면 secret이 한 번만 반환됩니다. 대상은 공개 HTTPS 443이어야 하며 HMAC 입력은 아래와 같습니다.

```
signed = X-Umm-Timestamp + "." + raw_request_body
X-Umm-Signature-256 = "sha256=" + hex(HMAC-SHA256(secret, signed))
```

도메인 변경과 활성 구독별 PostgreSQL outbox는 같은 트랜잭션에서 확정되며, 프로세스가 재시작되어도 워커가 대기 항목을 이어서 처리합니다. 워커는 정확한 delivery claim과 구독·소유 사용자·이벤트 공간·현재 membership을 실제 HTTP 응답까지 하나의 authorization lease로 잠급니다. 권한 회수·사용자 비활성화·구독 중지가 먼저 확정되면 캡처한 payload를 보내지 않고, 전달이 먼저 시작되면 정책 변경은 terminal 상태와 payload 삭제가 확정될 때까지 기다립니다. 이 lease는 일반 request pool 밖의 인스턴스당 최대 3개 연결을 사용합니다. 전달 시도는 at-least-once 방

식이므로 수신 측은 `X-Umm-Delivery` 를 먹등 키로 사용해 중복을 무시하고 timestamp 허용 시간도 함께 확인해야 합니다. terminal payload는 즉시 비워지고 delivery metadata는 30일 보존됩니다. 이벤트에는 `space.updated` , `note.*` , `edge.created` , `comment.*` , `member.*` , `dream.accepted` 가 있습니다.

안전한 재시도와 오류

Canvas의 메모·연결·댓글 생성/수정/삭제 요청에 8~128자의 `Idempotency-Key` 를 보내면 같은 사용자와 키의 성공 응답을 24시간 재생합니다. 서버는 method·경로·query·본문 fingerprint가 같은 요청만 재생하며, 다른 요청에 키를 재사용하면 409입니다. 처리 중 2분 lease는 handler가 살아 있는 동안 갱신되고, 아직 처리 중이면 `Retry-After` 가 포함된 425를 반환합니다. 도메인 변경·SSE·webhook outbox·먹등 성공 응답은 같은 PostgreSQL 트랜잭션으로 확정되므로 성공 응답 기록 실패가 mutation 중복을 만들지 않습니다. handler 실행 전 프로세스가 중단되면 lease 만료 후 같은 요청이 예약을 자동 재획득합니다. 오프라인 클라이언트는 한 논리 작업에 같은 키를 유지하고 `Retry-After` 이후 다시 시도해야 합니다. 장시간 AI 요청과 API key·웹훅 서명 key처럼 비밀을 한 번만 공개하는 요청은 `Idempotency-Key` 를 거부합니다.

오류 본문은 `application/problem+json` 이며 `type` , `title` , `status` , `detail` 을 제공합니다. 기존 클라이언트를 위해 `error` 도 같은 설명을 유지합니다. 접근 가능한 메모의 version 충돌만 409와 `clientVersion` , 최신 서버 `latest` 메모를 반환합니다. 메모가 삭제되었거나 공간 접근 권한을 잃은 경우에는 404로 종료합니다.

브라우저 오프라인 queue는 409 충돌과 408·425·429·5xx처럼 다시 시도할 수 있는 응답만 보존하고 일시 오류는 `Retry-After` 또는 기본 5초 뒤 자동 재시도합니다. 400·404 등 영구적인 client 오류는 사용자에게 사유를 알린 뒤 queue에서 제거해 무한 재시도를 막습니다. 메모를 볼 수는 있지만 공간 권한이 `edit` 에서 `view` 로 낮아진 `note-read-only` , 더는 댓글을 바꿀 수 없는 `comment-mutation-forbidden` 403은 일반 인증/권한 403과 구분해 해당 변경만 제거하고 이후 항목을 계속 동기화합니다. flush 도중 다른 탭이나 새 편집이 추가한 항목은 최신 queue와 ID 단위로 병합하고 탭 간 Web Lock으로 저장을 직렬화해 진행 중이던 snapshot이 덮어쓰지 않습니다.

⚡ 실시간 이벤트 스트림 (SSE)

- 경로: `GET /api/v1/spaces/{spaceID}/events`
- 프로토콜: Server-Sent Events (SSE)
- 이벤트 타입: `space-change`
- 용도: 타 사용자가 캔버스에서 포스트잇을 추가/수정/이동할 때 실시간 브로드캐스트 수신
- 재개: 각 이벤트의 `id` 가 `space_events.sequence` 입니다. 재연결 시 `Last-Event-ID` 헤더 또는 `?after=` 쿼리로 마지막 sequence를 전달하면 그 이후부터 이어받습니다.
- 전달 방식: 서버는 PostgreSQL `LISTEN/NOTIFY` 로 깨어나 즉시 전송합니다. 구독자 수와 무관하게 유휴 상태에서는 데이터베이스를 조회하지 않습니다. 수신 연결 상태가 바뀌면 열린 SSE를 즉시 깨우고 cursor 이후를 다시 조회한 뒤, 단절 중에는 1초 폴링으로 전환하고 복구 시 30초 safety net으로 돌아가므로 클라이언트가 알아차릴 필요는 없습니다.

3. Model Context Protocol (MCP) 엔드포인트

- 엔드포인트: `POST /mcp`
- 인증: `Authorization: Bearer <API_KEY>`

- 프로토콜: JSON-RPC 2.0 (Stateless HTTP POST)

제공 도구 목록 (Tools)

1. `list_spaces` : 접근 가능한 공간 목록 조회
2. `list_notes` : 지정된 공간의 생각 포스트잇 및 연결선 조회
3. `create_note` : 캔버스에 새로운 생각 포스트잇 생성
4. `connect_notes` : 두 생각 간의 연결선 생성
5. `search_notes` : 지정 공간의 생각 검색
6. `get_related_notes` : 오프라인 유사도 기반 연관 생각 조회
7. `list_clusters` : 생각 군집 조회
8. `list_dreams` : 출처와 검토 상태를 포함한 최근 Dream 조회

MCP 호출 예시 (JSON-RPC 2.0)

요청: 생각 포스트잇 생성

```
{
  "jsonrpc": "2.0",
  "id": "req-101",
  "method": "tools/call",
  "params": {
    "name": "create_note",
    "arguments": {
      "space_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "content": "MCP 엔드포인트를 통해 Cursor나 Claude에서 직접 생각을 붙이고 연결할 수 있습니다.",
      "color": "lavender",
      "x": 400,
      "y": 250
    }
  }
}
```

응답

```
{
  "jsonrpc": "2.0",
  "id": "req-101",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "생각 포스트잇이 성공적으로 생성되었습니다. (ID: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d)"
      }
    ]
  }
}
```